YARG Contributions — Roadmap

Canonical roadmap for both repos in this project (`yarg-song-server` and the `yarg` client fork). Status as of 2026-09-05.

Two tracks run in parallel. The **server track** is the #1 priority and is sequential — each phase depends on the one before it. The **client track** is independent and can absorb spare cycles at any time.

Phase 0 — Foundations ✅

Repos, remotes, mirrors and the format research that everything else is built on.

- [x] `fatalexception/yarg-song-server` and `fatalexception/yarg` created on Vault2 GitLab
- [x] Song-format research spec written (`docs/research/yarg-song-formats.md`)
- [x] Architecture decision recorded (`docs/ADR-001-server-architecture.md`)
- [x] Project instructions mirrored into `CLAUDE.md` so both machines pick them up
- [x] `yarg` imported from upstream server-side: 21 branches, 172 MB of repository and 180 MB of LFS objects, default branch set to `dev`
- [x] Public push mirrors to `github.com/Coffeehedake` configured and verified: `yarg-song-server` (GitLab mirror 10) and `yarg` (mirror 11), both one-way, all branches, divergent refs not kept. First sync 2026-09-05 16:44 ET, both `finished` with no error, and the GitHub side matches GitLab commit-for-commit.

Deliberately deferred: the `yarg` fork is *not* cloned locally. The project folder is inside the Syncthing `dev-projects` mesh, so a local clone replicates ~350 MB to r7 and Vault2 for a repo nothing touches until Phase 3. Clone it when Phase 3 starts:

```
cd "C:\dev\YARG - Open Source Contributions"
git clone https://gitlab.badassium.com/fatalexception/yarg.git yarg
cd yarg
git remote add upstream https://github.com/YARC-Official/YARG.git
git submodule update --init --recursive
git lfs pull
```

Exit criterion: both repos exist, both push to Vault2, both mirror to GitHub, and the format spec is committed. **Met 2026-09-05.**

`Coffeehedake/yarg` is a real GitHub *fork* of `YARC-Official/YARG`, not a plain repository, because that is the only shape GitHub accepts a pull request to upstream from.

Correction — LFS does propagate. An earlier revision of this roadmap claimed GitLab push mirroring drops LFS objects, and that the fork relationship was what made upstream's ~180 MB resolve. That was asserted from priors, not measured, and it is wrong.

The measurement: a throwaway GitLab project (**not** a fork, so no parent LFS storage to borrow from) with `*.png filter=lfs`, one fresh 1 MB object, push-mirrored to a throwaway **non-fork** GitHub repo. GitHub's LFS batch API returned a download action with no error, and the object downloaded from `github-cloud.githubusercontent.com` at exactly 1,048,576 bytes with a SHA-256 matching the OID. GitLab 18.11.3, git-lfs 3.7.1. Both throwaway repos were deleted afterwards.

So no special handling is needed when client work adds one of the five patterns YARG's `.gitattributes` tracks — `*.png`, `*.exr`, `*.jpg`, `*.fbx`, `*.ttf`. Which is just as well: almost any UI work in Phase 3 adds a `.png`.

The caveat that does survive: the mirror force-overwrites and the GitHub side is a fork, so anything done directly on GitHub is clobbered on the next sync. To open an upstream PR, create the branch on GitLab, let it mirror, open the PR from the mirrored branch, and leave it alone on GitHub after that.

Phase 1 — `yargsong`: the Go format library ✅

The server is worthless until Go can read and write what YARG reads and writes. This phase is pure library work with no network surface, which makes it the easiest phase to test exhaustively.

Build order (each step is independently testable):

1. **`song.ini` reader** — ✅ done (`internal/songini`). ~130 recognised keys with their types, and a deliberately lenient reader: UTF-8/UTF-16 BOMs, Latin-1 fallback so accented artist names survive, `[song]` / `[Song]` /no-header variance, trailing junk after the section bracket, values containing `=`, and malformed lines skipped rather than fatal. Two behaviours were checked against upstream rather than guessed: **duplicate keys — last wins** (upstream stores modifiers by plain dictionary assignment), and **keys and section names are lowercased** before lookup. One thing is still assumed and flagged in the source: the exact whitespace and multiple-`=` handling inside `YARGTextReader.ExtractModifierName`, which has not been read. The writer is step 5 below.

2. `.sng` **reader** — ✅ done (`internal/sng`). Implemented as an `fs.FS`, including synthesised directories, so the folder scanner and the `.sng` scanner share exactly one code path; `fstest.TestFS` enforces that. The mask-origin question the research doc flagged as unconfirmed is now settled from `SngFileStream.cs`: **the XOR index is the byte offset within each contained file, restarting at 0 per file**, not the absolute offset in the `.sng`. Upstream reaches the same result only because it decrypts whole 1 MB buffers and 1 MB is a multiple of the 256-byte table; tracking the real offset is equivalent and survives arbitrary read sizes. Malformed input is rejected with an error rather than a panic, which is tested.

Validated against the reference encoder, 2026-09-05. SngCli v0.3.0 (MIT, from `mdsitton/SngFileFormat` — the tool that defines the format) encoded a song folder we wrote, and our reader read the result: same chart bytes, same SHA-1, same metadata, and chunked streaming matching a whole-file read. The archive is committed at `internal/sng/testdata/reference-sngcli-v0.3.0.sng` so this stays a regression test rather than a thing that was true once. Until this, every `.sng` the reader had seen was produced by an encoder written from the same understanding as the reader — which proves only that the two agree with each other.

Two things the real file taught us that synthetic input could not:

- **Duplicate `song.ini` keys: last wins — confirmed independently.** A probe archive with `name = FIRST VALUE` … `name = SECOND VALUE` round-tripped through SngCli as `SECOND VALUE`, matching what we had concluded from reading `IniModifierCollection.cs`.
- **A file's extension can lie about its container.** SngCli emits audio under a `.mp3` name regardless of source format: our `song.wav` came back as `song.mp3`, byte-identical, RIFF header intact. We classify stems by name, as YARG does, so the scan is unaffected — but anything that later *decodes* audio must sniff the container, and our own writer must not reproduce this. There is a test pinning the observation.
- **Scanner** — ✅ done (`internal/scan`, `internal/catalog`). Walks a folder or a `.sng` through the same `fs.FS`, applies the chart-file priority order, classifies stems (all 14 standard, 5 clean and 4 explicit variants), resolves album art with the `cover` key overriding `album.<ext>`, finds background/video/preview, and emits the v1 catalog schema. Unrecognised files are carried in `assets.other` and every `song.ini` key is preserved in `raw_metadata`, so parse-tuning keys we do not model still survive a repack — losing those would change how a chart plays.

A test packs the same content both ways and asserts the folder and the `.sng` produce the same chart hash, metadata and parts. That is the ADR-001 "one code path" claim being enforced rather than merely intended.

Unknown intensity is `-1`, never `0`, for both an explicit `-1` and an absent key — "the charter rated this trivially easy" and "the chart did not say" must not collapse into the same value. Two `diff_*` keys (`diff_drums_real_ps`, `diff_keys_real_ps`) are deliberately left unmapped rather than guessed; the source says why.

`yarg-song-server scan <path>` walks a library and prints the catalog as JSON, so the parsers can be pointed at a real collection before anything sits behind an HTTP API. 4. **Identity** — ✅ done. `SHA1(chart file bytes)`, matching YARG's `HashWrapper`, plus a `package_hash` of our own (SHA-256 over the sorted `name:sha256` pairs) for same-chart-different-audio cases. Note the folder and `.sng` forms of one song have the same chart hash but *different* package hashes, because the folder carries `song.ini` as a file and the `.sng` carries it in the header — that is correct, and the test asserts it rather than papering over it. 5. `.sng` **writer** — ✅ done (`internal/sng/write.go`, `scan.PackDir`, `yarg-song-server pack <folder> <out.sng>`). Validated three ways, weakest first:

- **Our own round-trip** — proves only that our reader and writer agree.
- **The reference decoder.** `SngCli decode` extracted our archive with zero errors, and `notes.chart`, `album.png` and the audio all came back byte-identical to the source folder. Our archive of the same song is the same size as SngCli's, 8,806 bytes.
- **The client.** YARG v0.15.0 scanned 15 archives written by us and accepted **14**, reading every title correctly out of our metadata section — including Latin-1, UTF-16LE, a duplicate key resolving to `SECOND`, and an unknown key surviving the repack.

The one rejection was `No notes found`, and a **control settles it**: SngCli's own archive of the same source folder, scanned alongside ours in the same pass, was rejected with the identical error. The fixture's hand-written chart has no note events. As far as the client is concerned our writer is indistinguishable from the reference encoder.

Deliberate differences from the reference encoder: we do **not** rename audio to `.mp3` (SngCli does this regardless of the source container), and we refuse `song.ini` as a contained file rather than letting it disagree with the metadata section. Filenames are lowercased and collisions after lowercasing are refused, metadata keys are emitted lowercase because YARG matches `.sng` keys against a lowercase table without normalising them, and the header size is asserted against what was actually written — a mismatch there would silently corrupt every file offset. 6. **Chart preparsers** — ✅ done (`internal/chart`). Determines which parts and difficulties a chart actually contains, for both `notes.mid` and `notes.chart`. No timing, no sustains, no HOPO inference — the browse UI needs an instrument grid, and the client already has YARG.Core for everything else.

Built from documentation this time, not from source: TheNathannator's Guitar Game Chart Formats, the RBN/C3 documentation, FireFox2000000's `.chart` spec and the Elite Drums spec. The study is in `docs/research/chart-preparsing.md` with a citation per claim.

The governing rule is never to range-test a difficulty block. Every instrument family has non-playable numbers inside its own blocks, and a range test turns each into a phantom difficulty. Sixteen tests exist mostly to pin the traps the documentation names:

- force-strum and HOPO markers sit inside the block and are not notes;

- five-fret note 59 is a left-hand *animation* note unless an `ENHANCED_OPENS` event says otherwise — counting it unconditionally invents an Easy difficulty on a large share of Rock Band-derived charts;
- every Pro Keys track is *required* to carry a range-shift marker, so a test not strictly inside 48–72 reports even empty tracks as present;
- the Pro Keys animation tracks use an identical note range and differ only by name, so track names are matched whole-string, never as substrings;
- Elite Drums' modifier octave carries a disco-flip marker that exists for downcharting and can sit on a difficulty with no gems;
- a lone `HARM1` is the lead line, not a harmony arrangement.

Drum type is a heuristic because it has to be — 4-lane, Pro and 5-lane share one track — with `song.ini`'s `pro_drums` / `five_lane_drums` overriding, and both set at once recorded as the documented invalid state rather than silently resolved. Elite Drums downcharting is honoured: a song with only `PART ELITE_DRUMS` reports 4-lane, Pro and 5-lane as `derived`, because the client shows them.

The SMF reader is hand-rolled rather than a library, deliberately. Charts violate the MIDI spec in two documented ways — running status not reset after SysEx or meta events, and `0xFF` bytes inside SysEx — and a strict parser rejects files YARG plays fine.

Cross-checked against the oracle: the corpus case whose `notes.mid` YARG rejected with "No notes found" now reports zero parts from our preparser too. Same conclusion, independent route.

Exit criterion: a CLI that scans a real song library, emits a catalog, repacks to `.sng`, and YARG scans the repacked output with identical metadata and an identical hash. **Met 2026-09-05.**

`yarg-song-server scan <path>` and `yarg-song-server pack <folder> <out.sng>` do the first two. For the third: `SngCli` decoded our archives with every file byte-identical to the source, and YARG accepted 14 of the 15 archives we wrote — the one rejection being a fixture whose chart has no note events, proven by a control in which SngCli's own archive of the same folder was rejected identically.

And the parts we report now agree with the client's own verdict: on the 22-case corpus, **every song YARG rejects is one this scanner independently flags**, by three routes — no parts detected, `no_audio`, and `ultrastar_no_title`.

Explicitly out of scope, permanently: CON/mogg decryption, `songcache.bin` generation, `.milo_xbox` / `.png_xbox` decoding, `.yarground` inspection, full MIDI chart semantics.

Phase 2 — The server, and a sync client that needs no client changes

Two deliverables. The sync client is what makes the server *useful* before any client fork work exists, and it is the proof that the server is correct.

2a — `yarg-song-server`

- [x] **HTTP API** — `internal/httpapi`, documented in `docs/API.md`. Browse and search over the 12 attributes YARG itself sorts by, `GET /song/{chart_hash}.sng`, and a bulk `POST /api/v1/have` that takes the hashes a client holds and answers what it is missing.
- [x] **The index and the store** — `internal/library`, in memory and rebuilt at start; `internal/packcache` packs a loose folder to `.sng` on demand and caches it by package hash. Both decisions and their costs are in `docs/ADR-002-v1-store.md`.
- [x] **Sort parity with the client** — `internal/sortkey` reproduces YARG.Core's `SortString` (rich-text stripping, diacritic folding, whitespace collapse, article removal, character grouping, UTF-16 ordinal comparison) and `internal/library` orders by upstream's own comparer chains. Without this a browse list is internally consistent and unlike anything the player sees in the game.
- [ ] Ingest: loose folders, `.sng`, and `.zip` / `.7z` of a loose folder. Refuse RB packages with a clear message.
- [x] **Config file + sane defaults** — `internal/config`. `key = value` with `#` comments, the same names as the flags, precedence defaults < file < flags actually typed, and `--write-config` to print a commented example. An unknown setting is an error, not a warning. A file NAMED on the command line and missing is fatal; the conventional `./yarg-song-server.conf` simply not existing is the normal first run and is silent.
- [x] **CI** — `.gitlab-ci.yml`. `gofmt`, `go vet`, the suite, the suite again with `-race`, all six release targets, and an assertion that the Dockerfile's Go is not older than `go.mod` requires. See below for what this replaced.
- [ ] A bound or an eviction policy for the pack cache. It is content-keyed and therefore always safe to delete, which is why this is a finishing task and not a design question.
- [x] **The multi-arch container image** — `container-image` in `.gitlab-ci.yml`, pushing `linux/amd64` + `linux/arm64` to `registry.badassium.com/fatalexception/yarg-song-server`.

And it needs no emulation, which is the part worth remembering. The earlier plan here was to install qemu/binfmt on Vault2. That plan was wrong. Go cross-compiles natively, and the Dockerfile already pins its build stage with `FROM --platform=$BUILDPLATFORM`, so the toolchain runs at the host's own architecture and Go emits the arm64 binary itself. Emulation only enters if the build stage is pulled *for* the target platform, which that directive prevents. **Nothing on the Vault2 host was changed, and nothing should be.**

That correction came from the ShopStack session, who also established that `juniper-pi-deploy` — the "Pi framework" this was going to be cloned from — builds bootable SD-card images and publishes no container images at all. Same word, different problem.

Two things the job does that are not obvious:

- **It creates a `docker-container` buildx builder.** The default builder uses the `docker` driver, which cannot build more than one platform and cannot produce a manifest list at all. Without this step buildx fails with a message about the driver rather than anything that mentions multi-arch.
- **It verifies the result rather than trusting the exit code.** `ci/verify-multiarch.sh` asserts the manifest covers both platforms *and* reads the ELF machine type of the binary inside the arm64 image. An amd64 binary under an arm64 manifest entry passes every check that only reads exit codes, and then fails to start on the Pi. The check is structural rather than execution-based on purpose: running the arm64 binary would need the emulation this project deliberately does not have.

What CI replaced. This project had no `.gitlab-ci.yml`, so GitLab fell through to Auto DevOps. Pipeline #2216 — the only pipeline this project has ever run — failed three jobs with exit 127 trying to execute `/build/build.sh`, `/bin/herokuish` and `lsif-go`. Those are container-executor assumptions, and the runner is a **shell** executor on Vault2, which has none of them. A red pipeline nobody believes is worse than no pipeline, so the CI here brings its own pinned Go toolchain (SHA-256 verified before use, cached between runs) rather than depending on whatever happens to be installed on the host.

Measured end to end on 2026-09-05, against the 22-case corpus: the server indexed all 22 in 15 ms, `POST /have` from an empty client reported 22 missing, all 22 downloaded through `GET /song/{hash}.sng`, and re-scanning the downloaded archives gave 22 songs and 0 failures.

Then the oracle, which is the only test that means anything here: **YARG v0.15.0 scanned the 22 archives the server produced and accepted 20.** The two it refused were the two corpus cases built to be refused, and our own scanner flags both independently — `No notes found` against a preparser reporting no parts at all, and `No audio accompanying the chart file` against our `no_audio` issue. The standard holds on the served archives as well as on the folders.

One thing that run turned up, worth knowing before Phase 2b. Repacking a loose UltraStar folder whose `notes.txt` has no `#TITLE` makes YARG *accept* a song it refuses as a folder. As a folder the title has to come from the chart, and a missing `#TITLE` is fatal — "Name metadata not provided". Packed, the name lives in the `.sng` metadata section, which the packer fills from `song.ini`, so the song has a title and plays. Our scanner agrees with the client in both forms (`ultrastar_no_title` on the folder, no issue on the archive), so nothing here is broken — but it means "the server serves exactly what the folder was" is not quite true for this one format, and a sync client will show a song the player could not previously play.

2b — sync client

A small companion binary that pulls the server's library into an ordinary local songs folder. Unmodified YARG then just sees files. This ships real value in days, with zero risk to the client, and exercises the whole server end-to-end.

- [x] **`yarg-sync`** — `internal/syncclient` and `cmd/yarg-sync`, documented in `docs/SYNC-CLIENT.md`. Writes `<chart_hash>.sng` and nothing else, verifies every download by re-deriving identity from the bytes it received, resolves a shared chart hash deterministically, and leaves everything it did not write strictly alone — including under `-prune`, which is off by default. Built for all six release platforms.
- [x] **End-to-end coverage** — `internal/e2e` runs the real `httpapi.Server`, `library.Build` and `packcache` against the real client. Stubs prove one side's logic; only this catches the two sides disagreeing about the wire. All five tests were red-proofed by breaking the code they cover.
- [ ] Run it against the oracle: a real YARG install pointed at a folder `yarg-sync` filled. Until that is measured, "unmodified YARG just sees files" is a claim, not a result.
- [ ] Ship it on the Pi alongside the server and run the exit criterion for real.

Known, documented, not a defect: Windows Defender quarantines `cmd/yarg-sync` builds as `Trojan:Win32/Bearfoos.A!ml` — a machine-learning false positive on the shape of a small unsigned Go HTTP downloader. It blocks Windows developer builds, not CI, not the container image, and not the server binary. A code-signing certificate is the real fix. Details in `docs/SYNC-CLIENT.md`.

Exit criterion: a Pi on the LAN serves a shared library; two machines running stock YARG both see the same songs without either one being modified.

Phase 3 — Native remote song source in the client fork

Only now does the `yarg/` fork get touched. This is the upstream-facing work.

- Add an HTTP song source alongside local folders — browse, stream, cache.
- Touches YARG's song cache, so it must be designed with upstream in mind rather than bolted on.
- Upstream posture: `CONTRIBUTING.md` says nothing about networking or remote sources in any of its six tiers. That is an absence, not an endorsement — **open a discussion with YARC before building the PR**, and frame it as a content source, not a new game mode.
- PRs target `dev`. Never `master`.

Exit criterion: the fork can browse and play from a server without a sync step, and a discussion thread exists upstream.

Phase 4 — Modular features and the server config menu

The point at which the server stops being one feature and becomes a platform.

- Feature registry: each capability is opt-in and independently enable-able.
- Config UI in the server app, so a user turns on only what they want.
- Candidate modules: multi-user libraries and permissions, playlists/setlists shared across clients, scores and leaderboards, library health reporting.

Phase 5 — LLM chart generation

The long-term goal, and the phase most likely to move. It depends on every phase above working.

- **Start from the tools that already exist.** Help:Charting names two, both free, that do the first half of this: **Ultimate Vocal Remover (UVR)** separates a mix into bass, drums, vocals and other stems, and Spotify's **Basic Pitch** generates pitched MIDI from a vocal stem — a baseline vocals chart to work from. The novel part of this phase is the instrument charting and the difficulty reduction, not stem separation, and building either from scratch would be waste.
- Upload any song; auto-generate instrument and vocal parts across all difficulty levels.
- Produce a complete, working, **editable** package — generated charts are a starting point, not a final answer.
- **Distribution constraint, non-negotiable:** the chart/vocal tracks must be packageable and distributable *separately from the audio*, so charts can be shared without the music.
- Note what YARN's guidelines establish about this: **visual synchronisation is itself a derivative work**, which is why they refuse no-derivatives licences. A generated chart is a derivative of the song it was generated from, not a neutral companion to it — so separating chart from audio reduces the problem, and does not by itself dissolve it.
- Runs against the existing GPU arbitration on the home estate rather than a new stack.

Client track (independent of the server line)

Can be picked up at any time; does not block the server.

- **Controller compatibility** — official Rock Band and Guitar Hero instruments first. Upstream handles this through PlasticBand-Unity, so fixes may belong there rather than in YARG.
- **Graphics** — general rendering and visual improvements.
- Both are ordinary upstream contributions: fork, branch off `dev`, PR to `dev`.

Blockers

None.

Toolchain and credentials, as measured

The Coffeehedake GitHub PAT was rotated on 2026-09-05 and verified (`login=Coffeehedake` , scopes `repo` , `workflow` , `project` , `write:packages` , `delete:packages` , `audit_log`).

Go 1.27.0 was installed on ENG-1 on 2026-09-05, so the toolchain claim is now measured on the machine the work actually happens on: `gofmt` clean, `go vet` exit 0, `go test ./...` green, and all six release targets building — linux/amd64, linux/arm64, linux/armv7, darwin/amd64, darwin/arm64, windows/amd64.

It is a **per-user** install at `%LOCALAPPDATA%\Programs\go` , from the official zip with its SHA-256 verified against `go.dev/dl/?mode=json` before extraction, with `go\bin` and `%USERPROFILE%\go\bin` added to the user `Path` . Per-user rather than machine-wide on purpose: the MSI needs elevation, and an unattended `msiexec /qn` from a non-elevated session fails *silently* — the same failure mode that produced the Bambu Studio update loop. Nothing about this install needs elevation, and `GOPATH` / `GOCACHE` land in the user profile rather than in the Syncthing root. Promote it to a machine-wide install later if you want; nothing depends on where it lives.

mingw-w64 followed, for the race detector. `go test -race` needs cgo, and cgo needs a C compiler; without one ENG-1 reported `CGO_ENABLED=0` and `-race` failed outright with "requires cgo" rather than passing quietly. WinLibs GCC **16.2.0** (UCRT, POSIX threads, SEH) is now installed the same way — per-user at `%LOCALAPPDATA%\Programs\mingw64` , SHA-256 verified against the publisher's own `.sha256` before extraction, `bin` on the user `Path` . `go env CGO_ENABLED` now reports 1 and the suite is race-clean in about 16 seconds on the first run.

The extraction ran through a **scheduled task** rather than `Start-Process` , because a long child process started from a Cowork session dies with the process tree when a bridge call times out — that is what silently cancelled an earlier `winget` attempt mid-download. A scheduled task is detached from that tree and survives.

And the detector was proved able to go **red** before its green was believed: a throwaway module with eight goroutines incrementing a shared int returned `WARNING: DATA RACE` and exit 1.

Sequencing note

Phases 1 and 2 are the whole of the "#1 priority". Nothing in phases 3–5 should start before a stock YARG client is reading songs from a self-hosted server, because every later decision depends on what that turns out to actually require.